Chapter 2 Review Questions

1. The Web: HTTP; file transfer: FTP; remote login: Telnet; e-mail: SMTP; BitTorrent file sharing: BitTorrent protocol

2. Network architecture refers to the organization of the communication process into layers (e.g., the five-layer Internet architecture). Application architecture, on the other hand, is designed by an application developer and dictates the broad structure of the application (e.g., client-server or P2P).

3. The process which initiates the communication is the client; the process that waits to be contacted is the server.

4. In this case, client and server are roles, depending on the session the peer is involved in. Nothing prevents a peer to be both a client and a server simultaneously, for instance, as it downloads a file while also uploading some of its pieces (as in BitTorrent).

5. The IP address of the destination host and the port number of the socket in the destination process.

6. HTTP is an application layer protocol for transferring documents (i.e., web pages) and the objects referenced therein. The HTTP protocol only provides the functions to request and retrieve files. Another necessary component to realize a Web application is a document format that describes a hypertext. This allows a web browser to understand what objects need to be requested after a web page has been downloaded, for instance, images to be displayed next to some text. The standard format for the Web is the Hyper Text Markup Language (HTML).

7. One such example is remote word processing, for example, with Google docs. However, because Google docs runs over the Internet (using TCP), timing guarantees are not provided.

8. a) Reliable data transfer
   TCP provides a reliable byte-stream between client and server but UDP does not.

   b) A guarantee that a certain value for throughput will be maintained
   Neither

   c) A guarantee that data will be delivered within a specified amount of time
   Neither

   d) Confidentiality (via encryption)
   Neither

9. SSL operates at the application layer. The SSL socket takes unencrypted data from the application layer, encrypts it and then passes it to the TCP socket. If the application developer wants TCP to be enhanced with SSL, she has to include the SSL code in the application.

10. A protocol uses handshaking if the two communicating entities first exchange control packets before sending data to each other. SMTP uses handshaking at the application layer whereas HTTP does not.

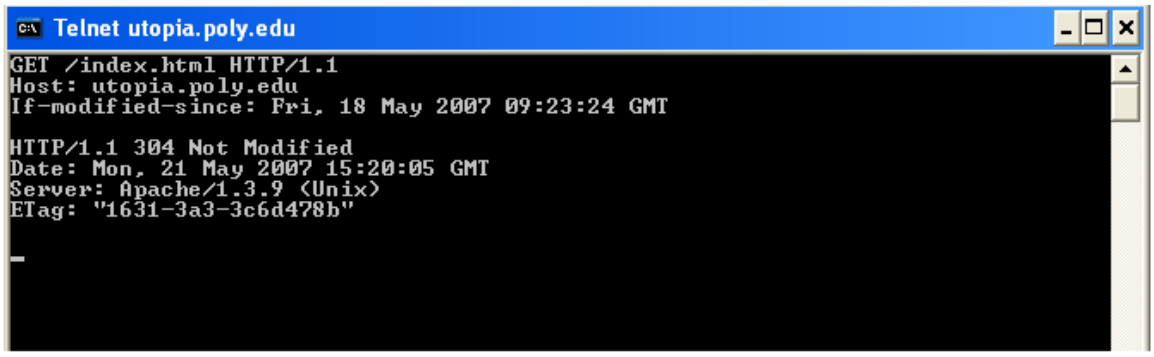
11. A protocol is called stateless when it does not keep any information about the clients during one or more communication sessions; otherwise, it is known as stateful. In other words, each request sent by a client using a stateless protocol can be interpreted independently of others. IMAP is not stateless (i.e., it is stateful), as it "remembers" user information (for instance, what messages have been read) across different sessions (see also Section 3 of RFC 3501). SMTP is stateful too (see Section 2.3.6 of RFC 5321).

12. As the HTTP protocol is stateless, the website needs to explicitly do something to identify users. One way is to use cookies as explained in Section 2.2.4. Cookies allow websites to track users across different sessions (i.e., when the browser is closed and re-opened). If it is enough to keep track of a user within a single session, the website can generate a unique ID at the server side and craft webpages so that such an ID is appended to all subsequent requests (for instance, at the end of the URLs).

13. Web caching can bring the desired content "closer" to the user, possibly to the same LAN to which the user's host is connected. Web caching can reduce the delay for all objects, even objects that are not cached, since caching reduces the traffic on links.

14. Telnet is not available in Windows 7 by default. to make it available, go to Control Panel, Programs and Features, Turn Windows Features On or Off, Check Telnet client. To start Telnet, in Windows command prompt, issue the following command
    > telnet webserverver 80

    where "webserver" is some webserver. After issuing the command, you have established a TCP connection between your client telnet program and the web server. Then type in an HTTP GET message. An example is given below:



Since the index.html page in this web server was not modified since Fri, 18 May 2007 09:23:34 GMT, and the above commands were issued on Sat, 19 May 2007, the server returned "304 Not Modified". Note that the first 4 lines are the GET message and header lines inputed by the user, and the next 4 lines (starting from HTTP/1.1 304 Not Modified) is the response from the web server.

15. There are no restrictions on the format of the HTTP body: it simply consists of the actual file in its original format. As a consequence, the HTTP body can also contain binary data, such as images. SMTP requires the body of an email message to be expressed a sequence of 7-bit ASCII characters. Arbitrary (possibly binary) data need to be encoded into 7-bit ASCII characters before they can be added to the body of an email message. Once the message is received, such data need to be decoded to get back the original content. Base64 is a popular scheme for encoding arbitrary data into 7-bit ASCII characters (see also RFC 4648).

16. The message is first sent from Alice's host to her mail server over HTTP. Alice's mail server then sends the message to Bob's mail server over SMTP. Bob then transfers the message from his mail server to his host over POP3.

17.

Received:	from 65.54.246.203 (EHLO bay0-omc3-s3.bay0.hotmail.com) (65.54.246.203) by mta419.mail.mud.yahoo.com with SMTP; Sat, 19 May 2007 16:53:51 -0700
Received:	from hotmail.com ([65.55.135.106]) by bay0-omc3-s3.bay0.hotmail.com with Microsoft SMTPSVC(6.0.3790.2668); Sat, 19 May 2007 16:52:42 -0700
Received:	from mail pickup service by hotmail.com with Microsoft SMTPSVC; Sat, 19 May 2007 16:52:41 -0700
Message-ID:	<BAY130-F26D9E35BF59E0D18A819AFB9310@phx.gbl>
Received:	from 65.55.135.123 by by130fd.bay130.hotmail.msn.com with HTTP; Sat, 19 May 2007 23:52:36 GMT
From:	"prithula dhungel" <prithuladhungel@hotmail.com>
To:	prithula@yahoo.com
Bcc:	
Subject:	Test mail
Date:	Sat, 19 May 2007 23:52:36 +0000
Mime-Version:	1.0
Content-Type:	Text/html; format=flowed
Return-Path:	prithuladhungel@hotmail.com

Figure: A sample mail message header

Received: This header field indicates the sequence in which the SMTP servers send and receive the mail message including the respective timestamps.

In this example there are 4 “Received:” header lines. This means the mail message passed through 5 different SMTP servers before being delivered to the receiver’s mail box. The last (forth) “Received:” header indicates the mail message flow from the SMTP server of the sender to the second SMTP server in the chain of servers. The sender’s SMTP server is at address 65.55.135.123 and the second SMTP server in the chain is by130fd.bay130.hotmail.msn.com.
The third “Received:” header indicates the mail message flow from the second SMTP server in the chain to the third server, and so on.
Finally, the first “Received:” header indicates the flow of the mail messages from the forth SMTP server to the last SMTP server (i.e. the receiver’s mail server) in the chain.

Message-id: The message has been given this number BAY130-F26D9E35BF59E0D18A819AFB9310@phx.gbl (by bay0-omc3-s3.bay0.hotmail.com. Message-id is a unique string assigned by the mail system when the message is first created.

From: This indicates the email address of the sender of the mail. In the given example, the sender is “prithuladhungel@hotmail.com”

To: This field indicates the email address of the receiver of the mail. In the example, the receiver is “prithula@yahoo.com”

Subject: This gives the subject of the mail (if any specified by the sender). In the example, the subject specified by the sender is “Test mail”

Date: The date and time when the mail was sent by the sender. In the example, the sender sent the mail on 19th May 2007, at time 23:52:36 GMT.

Mime-version: MIME version used for the mail. In the example, it is 1.0.

Content-type: The type of content in the body of the mail message. In the example, it is “text/html”.

Return-Path: This specifies the email address to which the mail will be sent if the receiver of this mail wants to reply to the sender. This is also used by the sender’s mail server for bouncing back undeliverable mail messages of mailer-daemon error messages. In the example, the return path is “prithuladhungel@hotmail.com”.

18. With the “download and keep” strategy, the email client retrieves the messages and stores them locally without deleting them from the server. Assume you download your email from your laptop, then disconnect and reconnect to the same email server from your smartphone. Also assume that this is the first time you connect on the smartphone and that you have not received any new emails in the meantime. The email client on the smartphone issues the list command and obtains the same list of the messages previously seen by the laptop email client. As there is no way to flag a

message as read on the server (as opposed to IMAP), the email client on the smartphone cannot tell whether messages have been read on (or even downloaded by) another client.

19. MX records are needed as they provide a service-specific DNS reply that applies to email addresses only. The same cannot be accomplished with a CNAME record. For instance, assume you own the yourbusiness.com domain and you want all email addresses in the form user@yourbusiness.com to reach mail.yourbusiness.com, while you want all web traffic to yourbusiness.com to reach www.yourbusiness.com. The only option you would have is to set both mail.yourbusiness.com and www.yourbusiness.com as aliases for yourbusiness.com, but this would not work as the former only hosts an email server and not a Web server.

20. A recursive DNS query allows the server to issue additional queries to other DNS servers before replying to the client. In contrast, an iterative DNS query allows the server to reply with a referral to another DNS server. The client has to explicitly query the DNS server in the referral to proceed with name resolution.

21. BitTorrent is much faster than the centralized client-server approach when the total upload capacity of the system (i.e., the denominator of the last term in Equation 2.2) is high. This happens when the sum of the upload rates of individual peers is much higher than us.

22. Recall that in BitTorrent, a peer picks a random peer and optimistically unchokes the peer for a short period of time. Therefore, Alice will eventually be optimistically unchoked by one of her neighbors, during which time she will receive chunks from that neighbor.

23. If the BitTorrent tracker becomes unavailable, no new downloads can be started. In fact, a peer cannot obtain the list of neighbors to download chunks from. However, ongoing downloads can still continue, as peers have a list of neighbors already. This list may become quickly outdated but downloads can progress at least for a short time.

24. One server placement philosophy is called Enter Deep, which enter deep into the access networks of Internet Service Providers, by deploying server clusters in access ISPs all over the world. The goal is to reduce delays and increase throughput between end users and the CDN servers. Another philosophy is Bring Home, which bring the ISPs home by building large CDN server clusters at a smaller number of sites and typically placing these server clusters in IXPs (Internet Exchange Points). This Bring Home design typically results in lower maintenance and management cost, compared with the enter-deep design philosophy.

25. Other than network-related factors, there are some important factors to consider, such as load-balancing (clients should not be directed to overload clusters), diurnal effects, variations across DNS servers within a network, limited availability of rarely

accessed video, and the need to alleviate hot-spots that may arise due to popular video content.
Reference paper:
Torres, Ruben, et al. "Dissecting video server selection strategies in the YouTube CDN." The 31st IEEE International Conference on. Distributed Computing Systems (ICDCS), 2011.

Another factor to consider is ISP delivery cost – the clusters may be chosen so that specific ISPs are used to carry CDN-to-client traffic, taking into account the different cost structures in the contractual relationships between ISPs and cluster operators.

26. With the UDP server, there is no welcoming socket, and all data from different clients enters the server through this one socket. With the TCP server, there is a welcoming socket, and each time a client initiates a connection to the server, a new socket is created. Thus, to support n simultaneous connections, the server would need *n+1* sockets.

27. For the TCP application, as soon as the client is executed, it attempts to initiate a TCP connection with the server. If the TCP server is not running, then the client will fail to make a connection. For the UDP application, the client does not initiate connections (or attempt to communicate with the UDP server) immediately upon execution

Chapter 2 Problems

Problem 1

a) F
b) T
c) F
d) F
e) F

Problem 2

SMS (Short Message Service) is a technology that allows the sending and receiving of text messages between mobile phones over cellular networks. One SMS message can contain data of 140 bytes and it supports languages internationally. The maximum size of a message can be 160 7-bit characters, 140 8-bit characters, or 70 16-bit characters. SMS is realized through the Mobile Application Part (MAP) of the SS#7 protocol, and the Short Message protocol is defined by 3GPP TS 23.040 and 3GPP TS 23.041. In addition, MMS (Multimedia Messaging Service) extends the capability of original text messages, and support sending photos, longer text messages, and other content.

iMessage is an instant messenger service developed by Apple. iMessage supports texts, photos, audios or videos that we send to iOS devices and Macs over cellular data network or WiFi. Apple's iMessage is based on a proprietary, binary protocol APNs (Apple Push Notification Service).

WhatsApp Messenger is an instant messenger service that supports many mobile platforms such as iOS, Android, Mobile Phone, and Blackberry. WhatsApp users can send each other unlimited images, texts, audios, or videos over cellular data network or WiFi. WhatsApp uses the XMPP protocol (Extensible Messaging and Presence Protocol).

iMessage and WhatsApp are different than SMS because they use data plan to send messages and they work on TCP/IP networks, but SMS use the text messaging plan we purchase from our wireless carrier. Moreover, iMessage and WhatsApp support sending photos, videos, files, etc., while the original SMS can only send text message. Finally, iMessage and WhatsApp can work via WiFi, but SMS cannot.

Problem 3

The browser first needs to retrieve the IP address corresponding to the server name in the URL. To do so, it contacts the local DNS server (with UDP on port 53) and issues a Type A query. Once it has obtained the IP address, the browser opens a connection (with TCP on port 80) with the web browser and sends an HTTP request in the form GET /about HTTP/1.1 followed by some headers (e.g., the User-agent header). The server replies with an HTTP response containing the HTML document in its body. If the webpage

includes other resources, such as images, the browser requests each of them through the same connection or with multiple connections, depending on its settings

Problem 4

a) The document request was http://gaia.cs.umass.edu/cs453/index.html. The Host : field indicates the server's name and /cs453/index.html indicates the file name.

b) The browser is running HTTP version 1.1, as indicated just before the first <cr><lf> pair.

c) The browser is requesting a persistent connection, as indicated by the Connection: keep-alive.

d) This is a trick question. This information is not contained in an HTTP message anywhere. So there is no way to tell this from looking at the exchange of HTTP messages alone. One would need information from the IP datagrams (that carried the TCP segment that carried the HTTP GET request) to answer this question.

e) Mozilla/5.0. The browser type information is needed by the server to send different versions of the same object to different types of browsers.

Problem 5

a) The status code of 200 and the phrase OK indicate that the server was able to locate the document successfully. The reply was provided on Tuesday, 07 Mar 2008 12:39:45 Greenwich Mean Time.

b) The document index.html was last modified on Saturday 10 Dec 2005 18:27:46 GMT.

c) There are 3874 bytes in the document being returned.

d) The first five bytes of the returned document are : <!doc. The server agreed to a persistent connection, as indicated by the Connection: Keep-Alive field

Problem 6

a) Persistent connections are discussed in section 8 of RFC 2616 (the real goal of this question was to get you to retrieve and read an RFC). Sections 8.1.2 and 8.1.2.1 of the RFC indicate that either the client or the server can indicate to the other that it is going to close the persistent connection. It does so by including the connection-token "close" in the Connection-header field of the http request/reply.
b) HTTP does not provide any encryption services.

c) (From RFC 2616) “Clients that use persistent connections should limit the number of simultaneous connections that they maintain to a given server. A single-user client SHOULD NOT maintain more than 2 connections with any server or proxy.”

d) Yes. (From RFC 2616) “A client might have started to send a new request at the same time that the server has decided to close the "idle" connection. From the server's point of view, the connection is being closed while it was idle, but from the client's point of view, a request is in progress.”

Problem 7

As no DNS records are cached at any of the servers, all interactions shown in Figures 2.19 and 2.20 are needed for both a. and b.

a. $RTT_l + 3 \cdot RTT_r$.

b. The same as point a.

c. RTT_l in both cases.

Problem 8

a)

$$RTT_1 + \cdots + RTT_n + 2RTT_o + 8 \cdot 2RTT_o$$
$$= 18RTT_o + RTT_1 + \cdots + RTT_n .$$

b)

$$RTT_1 + \cdots + RTT_n + 2RTT_o + 2 \cdot 2RTT_o$$
$$= 6RTT_o + RTT_1 + \cdots + RTT_n$$

c) Persistent connection with pipelining. This is the default mode of HTTP.

$$RTT_1 + \cdots + RTT_n + 2RTT_o + RTT_o$$
$$= 3RTT_o + RTT_1 + \cdots + RTT_n .$$

Persistent connection without pipelining, without parallel connections.

$$RTT_1 + \cdots + RTT_n + 2RTT_o + 8RTT_o$$
$$= 10RTT_o + RTT_1 + \cdots + RTT_n .$$

Problem 9

a) The time to transmit an object of size L over a link or rate R is L/R. The average time is the average size of the object divided by R:

Δ = (850,000 bits)/(15,000,000 bits/sec) = .0567 sec

The traffic intensity on the link is given by $\beta\Delta$=(16 requests/sec)(.0567 sec/request) = 0.907. Thus, the average access delay is (.0567 sec)/(1 - .907) ≈ .6 seconds. The total average response time is therefore .6 sec + 3 sec = 3.6 sec.

b) The traffic intensity on the access link is reduced by 60% since the 60% of the requests are satisfied within the institutional network. Thus the average access delay is (.0567 sec)/[1 – (.4)(.907)] = .089 seconds. The response time is approximately zero if the request is satisfied by the cache (which happens with probability .6); the average response time is .089 sec + 3 sec = 3.089 sec for cache misses (which happens 40% of the time). So the average response time is (.6)(0 sec) + (.4)(3.089 sec) = 1.24 seconds. Thus the average response time is reduced from 3.6 sec to 1.24 sec.

Problem 10

The total download time is:

a. $2 \cdot 100$ ms + $8 \cdot 10^3$ bits / 10^6 bits/s + $5 \cdot (2 \cdot 100$ ms + $4 \cdot 10^5$ bits / 10^6 bits/s) = 3.208 s

b. $2 \cdot 100$ ms + $3 \cdot (4 \cdot 10^5$ bits / 10^6 bits/s) = 1.4 s

c. $2 \cdot 100$ ms + $4 \cdot 10^5$ bits / 10^6 bits/s = 0.6 s

d. $2 \cdot 100$ ms + $8 \cdot 10^3$ bits / 10^6 bits/s + $5 \cdot (4 \cdot 10^5$ bits / 10^6 bits/s) = 2.208 s

Problem 11

The total download time for non-persistent HTTP with serial connections is:

$$2 \times RTT_{avg} + L_d / R + 2 \times N \times RTT_{avg} + \sum_{i=0}^{N-1} L_i / R$$

While that for a single persistent HTTP connection is:

$$2 \times RTT_{avg} + L_d / R + \sum_{i=0}^{N-1} L_i / R$$

Problem 12

Server.py

```
from socket import *
serverPort=12000
serverSocket=socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
while 1:
```

```
    sentence = connectionSocket.recv(1024)
    print      'From      Server:',      sentence,      '\n'
serverSocket.close()
```

Problem 13

The SMTP protocol is enough to transfer email messages between email servers. Thus, no mail access protocol is needed if a user has some kind of direct access to an SMTP server. There are two main scenarios here, both described by Figure 2.15. In the first one, Alice (Bob) runs an SMTP server on her (his) local PC and employs a user agent to read the messages directly from the server's mailbox on the same host. In the second scenario, Alice (Bob) employs a remote SMTP server and logs into the server to retrieve her (his) email through a user agent running on that server. This scenario is functionally equivalent to the first one, as the resources of the SMTP server are seen as "local" through the remote connection.

Problem 14

Recall that TCP provides a reliable data transfer services between a source and a destination process. Despite that, there are cases where TCP cannot help. For instance, one of the links on the path toward the destination can be down for long time. Moreover, the destination process may not be active, for instance, as the server has crashed or is being restarted due to hardware or software upgrades. When the destination server is unreachable, SMTP tries to retransmit an email message for a few times in an hour, and then increases the retransmission intervals (see also RFC 2821). Retries are attempted for a few days, after which the sender is notified that the message could not be delivered.

Problem 15

MTA stands for Mail Transfer Agent. A host sends the message to an MTA. The message then follows a sequence of MTAs to reach the receiver's mail reader. We see that this spam message follows a chain of MTAs. An honest MTA should report where it receives the message. Notice that in this message, `"asusus-4b96 ([58.88.21.177])"` does not report from where it received the email. Since we assume only the originator is dishonest, so `"asusus-4b96 ([58.88.21.177])"` must be the originator.

Problem 16

The SRV record allows us to request the address of a server based on its domain and a specific service/protocol combination. It can be seen as a generalization of the MX record (which only refers to email addresses) to diverse types of services.

Problem 17

a) **C:** dele 1
C: retr 2
S: (blah blah …
S: ……..blah)
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off

b) **C:** retr 2
S: blah blah …
S: ……..blah
S: .
C: quit
S: +OK POP3 server signing off

b) **C:** list
S: 1 498
S: 2 912
S: .
C: retr 1
S: blah …..
S: ….blah
S: .
C: retr 2
S: blah blah …
S: ……..blah
S: .
C: quit
S: +OK POP3 server signing off

Problem 18

a) For a given input of domain name (such as ccn.com), IP address or network administrator name, the *whois* database can be used to locate the corresponding registrar, whois server, DNS server, and so on.

b) NS4.YAHOO.COM from www.register.com; NS1.MSFT.NET from ww.register.com

c) *Local Domain: www.mindspring.com*
Web servers : www.mindspring.com
207.69.189.21, 207.69.189.22,
207.69.189.23, 207.69.189.24,

207.69.189.25, 207.69.189.26, 207.69.189.27,
207.69.189.28
Mail Servers : mx1.mindspring.com (207.69.189.217)
mx2.mindspring.com (207.69.189.218)
mx3.mindspring.com (207.69.189.219)
mx4.mindspring.com (207.69.189.220)
Name Servers: itchy.earthlink.net (207.69.188.196)
scratchy.earthlink.net (207.69.188.197)

www.yahoo.com
Web Servers: www.yahoo.com (216.109.112.135, 66.94.234.13)
Mail Servers: a.mx.mail.yahoo.com (209.191.118.103)
b.mx.mail.yahoo.com (66.196.97.250)
c.mx.mail.yahoo.com (68.142.237.182, 216.39.53.3)
d.mx.mail.yahoo.com (216.39.53.2)
e.mx.mail.yahoo.com (216.39.53.1)
f.mx.mail.yahoo.com (209.191.88.247, 68.142.202.247)
g.mx.mail.yahoo.com (209.191.88.239, 206.190.53.191)
Name Servers: ns1.yahoo.com (66.218.71.63)
ns2.yahoo.com (68.142.255.16)
ns3.yahoo.com (217.12.4.104)
ns4.yahoo.com (68.142.196.63)
ns5.yahoo.com (216.109.116.17)
ns8.yahoo.com (202.165.104.22)
ns9.yahoo.com (202.160.176.146)

www.hotmail.com
Web Servers: www.hotmail.com (64.4.33.7, 64.4.32.7)

Mail Servers: mx1.hotmail.com (65.54.245.8, 65.54.244.8, 65.54.244.136)
mx2.hotmail.com (65.54.244.40, 65.54.244.168, 65.54.245.40)
mx3.hotmail.com (65.54.244.72, 65.54.244.200, 65.54.245.72)
mx4.hotmail.com (65.54.244.232, 65.54.245.104, 65.54.244.104)

Name Servers: ns1.msft.net (207.68.160.190)
ns2.msft.net (65.54.240.126)
ns3.msft.net (213.199.161.77)
ns4.msft.net (207.46.66.126)
ns5.msft.net (65.55.238.126)

d) The yahoo web server has multiple IP addresses
www.yahoo.com (216.109.112.135, 66.94.234.13)

e) The address range for Polytechnic University: 128.238.0.0 – 128.238.255.255

f) An attacker can use the *whois* database and nslookup tool to determine the IP address ranges, DNS server addresses, etc., for the target institution.

g) By analyzing the source address of attack packets, the victim can use whois to obtain information about domain from which the attack is coming and possibly inform the administrators of the origin domain.

Problem 19

a) The following delegation chain is used for gaia.cs.umass.edu
a.root-servers.net
E.GTLD-SERVERS.NET
ns1.umass.edu(authoritative)

First command:
dig +norecurse @a.root-servers.net any gaia.cs.umass.edu

;; AUTHORITY SECTION:

edu.	172800	IN	NS	E.GTLD-SERVERS.NET.
edu.	172800	IN	NS	A.GTLD-SERVERS.NET.
edu.	172800	IN	NS	G3.NSTLD.COM.
edu.	172800	IN	NS	D.GTLD-SERVERS.NET.
edu.	172800	IN	NS	H3.NSTLD.COM.
edu.	172800	IN	NS	L3.NSTLD.COM.
edu.	172800	IN	NS	M3.NSTLD.COM.
edu.	172800	IN	NS	C.GTLD-SERVERS.NET.

Among all returned edu DNS servers, we send a query to the first one.
dig +norecurse @E.GTLD-SERVERS.NET any gaia.cs.umass.edu

umass.edu.	172800	IN	NS	ns1.umass.edu.
umass.edu.	172800	IN	NS	ns2.umass.edu.
umass.edu.	172800	IN	NS	ns3.umass.edu.

Among all three returned authoritative DNS servers, we send a query to the first one.
dig +norecurse @ns1.umass.edu any gaia.cs.umass.edu

gaia.cs.umass.edu.	21600	IN	A	128.119.245.12

b) The answer for google.com could be:
a.root-servers.net
E.GTLD-SERVERS.NET
ns1.google.com(authoritative)

Problem 20

Without a proxy, all clients need to reach the origin server on the Internet and share the bandwidth of the bottleneck link. As a consequence, the total transfer time is $2 \times RTT + N \times L / R_b$. If a proxy is installed, the clients share the bandwidth on the local link with the proxy. Thus, the total transfer time is $2 \times RTT + L / R_b + N \times L / R_l$. The file transfer is faster with the proxy when $R_l > R_l \times N / (N - 1)$.

Problem 21

Yes, we can use dig to query that Web site in the local DNS server.
For example, "dig cnn.com" will return the query time for finding cnn.com. If cnn.com was just accessed a couple of seconds ago, an entry for cnn.com is cached in the local DNS cache, so the query time is 0 msec. Otherwise, the query time is large.

Problem 22

For calculating the minimum distribution time for client-server distribution, we use the following formula:

$$D_{cs} = max \{NF/u_s, F/d_{min}\}$$

Similarly, for calculating the minimum distribution time for P2P distribution, we use the following formula:

$$D_{P2P} = max\{F/u_s, F/d_{min}, NF/(u_s + \sum_{i=1}^{N} u_i)\}$$

Where, F = 15 Gbits = 15 * 1024 Mbits
u_s = 30 Mbps
$d_{min} = d_i$ = 2 Mbps

Note, 300Kbps = 300/1024 Mbps.

Client Server

		N		
		10	**100**	**1000**
	300 Kbps	7680	51200	512000
u	**700 Kbps**	7680	51200	512000
	2 Mbps	7680	51200	512000

Peer to Peer

		N		
		10	**100**	**1000**

	300 Kbps	7680	25904	47559
u	**700 Kbps**	7680	15616	21525
	2 Mbps	7680	7680	7680

Problem 23

a) Consider a distribution scheme in which the server sends the file to each client, in parallel, at a rate of a rate of u_s/N. Note that this rate is less than each of the client's download rate, since by assumption $u_s/N \leq d_{\min}$. Thus each client can also receive at rate u_s/N. Since each client receives at rate u_s/N, the time for each client to receive the entire file is $F/(u_s/N) = NF/ u_s$. Since all the clients receive the file in NF/ u_s, the overall distribution time is also NF/ u_s.

b) Consider a distribution scheme in which the server sends the file to each client, in parallel, at a rate of $d_{\min}$. Note that the aggregate rate, $N d_{\min}$, is less than the server's link rate u_s, since by assumption $u_s/N \geq d_{\min}$. Since each client receives at rate $d_{\min}$, the time for each client to receive the entire file is $F/ d_{\min}$. Since all the clients receive the file in this time, the overall distribution time is also $F/ d_{\min}$.

c) From Section 2.6 we know that

$D_{CS} \geq \max \{NF/u_s, F/d_{\min}\}$ (Equation 1)

Suppose that $u_s/N \leq d_{\min}$. Then from Equation 1 we have $D_{CS} \geq NF/u_s$. But from (a) we have $D_{CS} \leq NF/u_s$. Combining these two gives:

$D_{CS} = NF/u_s$ when $u_s/N \leq d_{\min}$. (Equation 2)

We can similarly show that:

$D_{CS} = F/d_{\min}$ when $u_s/N \geq d_{\min}$ (Equation 3).

Combining Equation 2 and Equation 3 gives the desired result.

Problem 24

a) Define u = u1 + u2 + ….. + uN. By assumption

$u_s <= (u_s + u)/N$ Equation 1

Divide the file into N parts, with the ith part having size $(u_i/u)F$. The server transmits the ith part to peer i at rate $r_i = (u_i/u)u_s$. Note that $r_1 + r_2 + ….. + r_N = u_s$, so that the aggregate server rate does not exceed the link rate of the server. Also have each peer i

forward the bits it receives to each of the *N-1* peers at rate r_i. The aggregate forwarding rate by peer i is $(N-1)r_i$. We have

$$(N-1)r_i = (N-1)(u_s u_i)/u <= u_i,$$

where the last inequality follows from Equation 1. Thus the aggregate forwarding rate of peer i is less than its link rate u_i.

In this distribution scheme, peer i receives bits at an aggregate rate of

$$r_i + \sum_{j<>i} r_j = u_s$$

Thus each peer receives the file in F/u_s.

b) Again define $u = u_1 + u_2 + ... + u_N$. By assumption

$u_s >= (u_s + u)/N$ Equation 2

Let $r_i = u_i/(N-1)$ and
$r_{N+1} = (u_s - u/(N-1))/N$

In this distribution scheme, the file is broken into *N+1* parts. The server sends bits from the ith part to the ith peer (i = *1, ..., N*) at rate r_i. Each peer i forwards the bits arriving at rate r_i to each of the other N-1 peers. Additionally, the server sends bits from the $(N+1)^{st}$ part at rate r_{N+1} to each of the *N* peers. The peers do not forward the bits from the $(N+1)^{st}$ part.

The aggregate send rate of the server is

$$r_1 + ... + r_N + N\, r_{N+1} = u/(N-1) + u_s - u/(N-1) = u_s$$

Thus, the server's send rate does not exceed its link rate. The aggregate send rate of peer i is

$$(N-1)r_i = u_i$$

Thus, each peer's send rate does not exceed its link rate.
In this distribution scheme, peer i receives bits at an aggregate rate of

$$r_i + r_{N+1} + \sum_{j<>i} rj = u/(N-1) + (u_s - u/(N-1))/N = (u_s + u)/N$$

Thus each peer receives the file in $NF/(u_s+u)$.

(For simplicity, we neglected to specify the size of the file part for i = 1, …., N+1. We now provide that here. Let $\Delta = (u_s+u)/N$ be the distribution time. For i = 1, …, N, the ith file part is $F_i = r_i \Delta$ bits. The $(N+1)^{st}$ file part is $F_{N+1} = r_{N+1} \Delta$ bits. It is straightforward to show that $F_1 + \ldots.. + F_{N+1} = F$.)

c) The solution to this part is similar to that of 17 (c). We know from section 2.6 that

$$D_{P2P} >= max\{F/u_s, NF/(u_s + u)\}$$

Combining this with a) and b) gives the desired result.

Problem 25

Yes. His first claim is possible, as long as there are enough peers staying in the swarm for a long enough time. Bob can always receive data through optimistic unchoking by other peers.

His second claim is also true. He can run a client on each host, let each client "free-ride," and combine the collected chunks from the different hosts into a single file. He can even write a small scheduling program to make the different hosts ask for different chunks of the file. This is actually a kind of Sybil attack in P2P networks.

Problem 26

a. N files, under the assumption that we do a one-to-one matching by pairing video versions with audio versions in a decreasing order of quality and rate.

b. 2N files.

Problem 27

a) If you run TCPClient first, then the client will attempt to make a TCP connection with a non-existent server process. A TCP connection will not be made.

b) UDPClient doesn't establish a TCP connection with the server. Thus, everything should work fine if you first run UDPClient, then run UDPServer, and then type some input into the keyboard.

c) If you use different port numbers, then the client will attempt to establish a TCP connection with the wrong process or a non-existent process. Errors will occur.

Problem 28

In the original program, UDPClient does not specify a port number when it creates the socket. In this case, the code lets the underlying operating system choose a port number. With the additional line, when UDPClient is executed, a UDP socket is created with port number 5432 .

UDPServer needs to know the client port number so that it can send packets back to the correct client socket. Glancing at UDPServer, we see that the client port number is not "hard-wired" into the server code; instead, UDPServer determines the client port number by unraveling the datagram it receives from the client. Thus UDP server will work with any client port number, including 5432. UDPServer therefore does not need to be modified.

Before:

Client socket = x (chosen by OS)
Server socket = 9876

After:

Client socket = 5432

Problem 29

Yes, you can configure many browsers to open multiple simultaneous connections to a Web site. The advantage is that you will you potentially download the file faster. The disadvantage is that you may be hogging the bandwidth, thereby significantly slowing down the downloads of other users who are sharing the same physical links.

Problem 30

For an application such as remote login (telnet and ssh), a byte-stream oriented protocol is very natural since there is no notion of message boundaries in the application. When a user types a character, we simply drop the character into the TCP connection.
In other applications, we may be sending a series of messages that have inherent boundaries between them. For example, when one SMTP mail server sends another SMTP mail server several email messages back to back. Since TCP does not have a mechanism to indicate the boundaries, the application must add the indications itself, so that receiving side of the application can distinguish one message from the next. If each message were instead put into a distinct UDP segment, the receiving end would be able to distinguish the various messages without any indications added by the sending side of the application.

Problem 31

As Netflix has its own servers deployed at both ISPs and IXPs, it employs a combination of the "enter deep" and "bring home" placement strategies. The number of servers

deployed at each location varies, but usually more servers are located at the IXPs and they may contain the whole Netflix library. The content is updated on the servers according to a push strategy. This allows Netflix to schedule video transfers to the servers in its CDN during the hours with less traffic, thus avoiding congestion.